

Divisão-e-conquista

Aula 4

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Introdução
- 2 Segmento de soma máxima
- 3 Método da substituição
- 4 Método da árvore de recursão
- 5 Método mestre
- 6 Exercícios

MERGE SORT

- ▶ na aula 2, vimos que o MERGE SORT é um exemplo do método da divisão-e-conquista

Divisão-e-conquista

Divida o problema em um número de subproblemas que são instâncias menores do mesmo problema

Conquiste os subproblemas solucionando-os recursivamente; se os tamanhos dos subproblemas são suficientemente pequenos, solucione os subproblemas de maneira fácil

Combine as soluções dos subproblemas na solução para o problema original

- ▶ quando um subproblema é grande o suficiente para solucioná-lo recursivamente, o chamamos de **caso recursivo**; caso contrário, é chamado **caso base**

Recorrências

- ▶ recorrências e o paradigma da divisão-e-conquista andam juntos porque elas são uma forma natural de caracterizar o tempo de execução de algoritmos de divisão-e-conquista
- ▶ uma **recorrência** é uma (in)equação que descreve uma função em termos de seu valor sobre entradas menores
- ▶ exemplos:
 - ▶ $T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 2T(n/2) + \Theta(n), & \text{se } n > 1. \end{cases}$
 - ▶ $T(n) = T(2n/3) + T(n/3) + \Theta(n).$
 - ▶ $T(n) = T(n-1) + \Theta(1).$

Recorrências

- ▶ veremos 3 métodos para solução de recorrências:
 - ▶ método da substituição
 - ▶ método da árvore de recursão
 - ▶ método mestre

Questões técnicas sobre recorrências

- ▶ na prática, muitas vezes negligenciamos alguns detalhes quando escrevemos e resolvemos recorrências
- ▶ por exemplo, quando chamamos o MERGESORT sobre n itens de entrada e n é ímpar, os subproblemas terão tamanhos $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$
- ▶ tecnicamente, o tempo de execução de pior caso do MERGESORT é na realidade

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n), & \text{se } n > 1. \end{cases}$$

- ▶ além disso, negligenciamos os casos pequenos, assumindo que $T(n) = \Theta(1)$ para n pequeno

Motivação

- ▶ suponha uma grandeza que varia com o tempo, aumentando ou diminuindo uma vez por dia, de maneira irregular
- ▶ dado o registro das variações diárias desta grandeza, ao longo de um ano, queremos encontrar um intervalo de tempo em que a variação acumulada tenha sido máxima

Definições

- ▶ um **segmento** de um vetor $A[p..r]$ é qualquer subvetor da forma $A[i..k]$, com $p \leq i \leq k \leq r$
- ▶ a **soma** de um segmento $A[i..k]$ é o valor $A[i] + A[i+1] + \dots + A[k]$
- ▶ a **solidez** de um vetor $A[p..r]$ é a soma de um segmento de soma máxima

Problema computacional

Problema do segmento de soma máxima

Dado um vetor $A[p..r]$ de números inteiros, com $r - p + 1 \geq 1$, calcular a solidez de $A[p..r]$

Exemplo

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

Exemplo

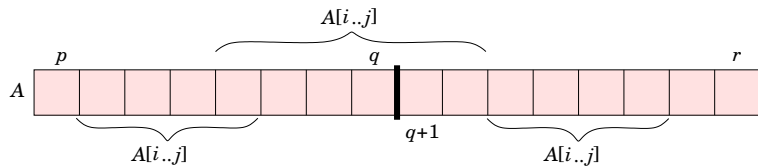
A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

Divisão-e-conquista

- ▶ para calcular a solidez de um vetor $A[p..r]$ podemos usar o método ingênuo ou da força-bruta
- ▶ também podemos usar o método da divisão-e-conquista
- ▶ observe que podemos dividir o vetor $A[p..r]$ ao meio, obtendo dois subvetores $A[p..q]$ e $A[q+1..r]$, com $q = \lfloor (p+r)/2 \rfloor$
- ▶ além disso, qualquer subvetor $A[i..j]$ do vetor $A[p..r]$ deve cair exatamente em um dos seguintes locais:
 - ▶ inteiramente no subvetor $A[p..q]$, com $p \leq i \leq j \leq q$,
 - ▶ inteiramente no subvetor $A[q+1..r]$, com $q < i \leq j \leq r$, ou
 - ▶ cruzando o ponto médio q , de tal forma que $p \leq i \leq q < j \leq r$

Exemplo



Algoritmo

SOLIDEZII(A, p, r)

01. **se** $p = r$

02. **devolva** $A[p]$

03. **senão**

04. $q \leftarrow \lfloor (p+r)/2 \rfloor$

05. $x \leftarrow \text{SOLIDEZII}(A, p, q)$

06. $y \leftarrow \text{SOLIDEZII}(A, q+1, r)$

07. $z \leftarrow \text{SOLIDEZ-MEIO}(A, p, q, r)$

08. **devolva** $\max(x, y, z)$

Algoritmo

SOLIDEZ-MEIO(A, p, q, r)

```

01.   $z' \leftarrow s \leftarrow A[q]$ 
02.  para  $i \leftarrow q - 1$  até  $p$  passo  $-1$ 
03.       $s \leftarrow s + A[i]$ 
04.      se  $s > z'$ 
05.           $z' \leftarrow s$ 
06.   $z'' \leftarrow s \leftarrow A[q + 1]$ 
07.  para  $j \leftarrow q + 2$  até  $r$ 
08.       $s \leftarrow s + A[j]$ 
09.      se  $s > z''$ 
10.           $z'' \leftarrow s$ 
11.  devolva  $z' + z''$ 

```

Exemplo

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

Exemplo

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

Análise do algoritmo

- ▶ o tempo de execução do algoritmo SOLIDEZ-MEIO é $\Theta(n)$, se consideramos que $A[p..r]$ tem $n = r - p + 1$ elementos
- ▶ dessa forma, o tempo de execução $T(n)$ do algoritmo SOLIDEZII pode ser determinado pela seguinte recorrência:

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 2T(n/2) + \Theta(n), & \text{se } n > 1. \end{cases}$$

- ▶ esta recorrência é idêntica à recorrência do MERGESORT
- ▶ assim, $T(n) = \Theta(n \lg n)$
- ▶ o problema do segmento de soma máxima pode ser resolvido usando a técnica de **programação dinâmica** em tempo $\Theta(n)$

Ideia geral

- ▶ “advinhe” a forma da solução
- ▶ use indução para encontrar as constantes e prove que a solução está correta

Exemplo

Seja $T(n) = 2T(\lfloor n/2 \rfloor) + n$ uma recorrência. Advinhamos que a solução é $O(n \lg n)$. O método da substituição exige que provemos que $T(n) \leq cn \lg n$ para uma constante $c > 0$. Começamos supondo que esse limitante está correto para todo $m < n$ e assim $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$, já que $\lfloor n/2 \rfloor < n$ para todo $n > 0$. Dessa forma,

$$\begin{aligned}
 T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
 &\leq 2\left(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)\right) + n \\
 &\leq cn \lg(n/2) + n \\
 &= cn \lg n - cn + n \\
 &\leq cn \lg n,
 \end{aligned}$$

para qualquer constante $c \geq 1$

Exemplo

Agora temos de provar que a solução está correta também para os casos pequenos. Fazemos isso mostrando que os casos pequenos são adequados como casos base da prova indutiva. Dessa forma, devemos mostrar que podemos escolher uma constante c grande o suficiente tal que o limitante $T(n) \leq cn \lg n$ funciona também para os casos pequenos. Suponha que $T(1) = 1$ é o único “caso pequeno” da recorrência. Então, para $n = 1$, o limitante $T(n) \leq cn \lg n$ fornece $T(1) \leq c1 \lg 1 = 0$, que não concorda com $T(1) = 1$ e o caso base da recorrência não funciona.

Exemplo

Lembre-se que temos que provar que a recorrência $T(n) = 2T(\lfloor n/2 \rfloor) + n$ é tal que $T(n) \leq cn \lg n$ para $n \geq n_0$, onde n_0 é uma constante que **temos de escolher**. Eliminamos então a condição $T(1) = 1$ da prova indutiva, observando que para $n > 3$ a recorrência não depende de $T(1)$. Dessa forma, podemos usar $T(2)$ e $T(3)$ como casos base da prova indutiva fazendo $n_0 = 2$. Note que há uma distinção entre caso base da recorrência, $n = 1$, e casos base da prova indutiva, $n = 2$ e $n = 3$. Com $T(1) = 1$, obtemos $T(2) = 4$ e $T(3) = 5$. Podemos agora completar a prova indutiva que $T(n) \leq cn \lg n$ para alguma constante $c \geq 1$ escolhendo c grande suficiente tal que $T(2) \leq c2 \lg 2$ e $T(3) \leq c3 \lg 3$. Qualquer escolha de $c \geq 2$ é suficiente para que os casos base $n = 2$ e $n = 3$ funcionem.

Como fazer boas escolhas

- ▶ não há uma forma geral de adivinhar soluções corretas para recorrências
- ▶ adivinhar uma solução depende de experiência e criatividade
- ▶ se a recorrência é similar a alguma que você viu antes, então chutar uma solução similar é razoável:

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n,$$

podemos chutar que $T(n) = O(n \lg n)$

- ▶ uma outra forma é provar limitantes grandes, inferior e superior, sobre a recorrência e então ir reduzindo a faixa de incerteza

Sutilezas

- ▶ algumas vezes você pode fazer um chute certo de um limitante assintótico da solução de uma recorrência, mas a matemática falha na prova da indução
- ▶ em geral, isso significa que a suposição indutiva não é forte o suficiente para provar o limitante
- ▶ podemos revisar o chute, quase sempre subtraindo um termo de pequena ordem

Sutilezas

Por exemplo, considere a recorrência $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$.
 Advinhamos que a solução é $T(n) = O(n)$ e tentamos mostrar que $T(n) \leq cn$ para uma escolha apropriada da constante c . Substituindo nosso chute na recorrência, temos

$$\begin{aligned} T(n) &\leq c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 \\ &= cn + 1. \end{aligned}$$

o que **não** implica que $T(n) \leq cn$ para qualquer escolha de c .

Sutilezas

Podemos tentar um chute mais alto, digamos $T(n) = O(n^2)$. Apesar disso, o chute inicial $T(n) = O(n)$ está correto. Mas para provar que ele está correto, devemos estabelecer uma hipótese indutiva mais forte.

Importante! A indução matemática não funciona, a menos que você prove a forma exata da hipótese de indução. No exemplo, podemos transpor essa dificuldade *subtraindo* um termo de pequena ordem do chute anterior. Isto é, fazendo $T(n) \leq cn - d$, onde $d \geq 0$ é uma constante. Assim,

$$\begin{aligned} T(n) &\leq (c\lfloor n/2 \rfloor - d) + (c\lceil n/2 \rceil - d) + 1 \\ &= cn - 2d + 1 \\ &\leq cn - d, \end{aligned}$$

para $d \geq 1$.

Evite erros!

Não é difícil errar quando se usa notação assintótica. Por exemplo, na recorrência $T(n) = 2T(\lfloor n/2 \rfloor) + n$ podemos falsamente “provar” que $T(n) = O(n)$, adivinhando que $T(n) \leq cn$ e argumentando que

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor) + n \\ &\leq cn + n \\ &= O(n), \end{aligned}$$

já que c é uma constante. O erro é que não provamos a **forma exata** da hipótese de indução, isto é, $T(n) \leq cn$. Temos que explicitamente provar que $T(n) \leq cn$ quando queremos mostrar que $T(n) = O(n)$.

Mudança de variáveis

Algumas vezes, um pouco de manipulação algébrica pode fazer uma recorrência desconhecida se tornar similar a uma vista anteriormente. Por exemplo, considere

$$T(n) = 2T(\sqrt{n}) + \lg n.$$

Podemos simplificar essa recorrência com uma mudança de variáveis. Por conveniência, não vamos nos preocupar com arredondamento de valores. Fazendo $m = \lg n$, temos

$$T(2^m) = 2T(2^{m/2}) + m.$$

Mudança de variáveis

Agora, podemos fazer $S(m) = T(2^m)$ e produzir a nova recorrência

$$S(m) = 2S(m/2) + m,$$

que já conhecemos. Assim, $S(m) = O(m \lg m)$. Voltando à fórmula original, temos

$$T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n).$$

Ideia geral

- ▶ no método da substituição, um bom chute pode ser difícil
- ▶ desenhar uma árvore de recursão, como fizemos na aula 2 para a recorrência do MERGESORT, é uma boa forma de obter um bom chute
- ▶ em uma **árvore de recursão** cada vértice representa o custo de um subproblema unitário, no conjunto de chamadas da função recursiva
- ▶ somamos o custo em cada nível da árvore para obter os custos por nível e então somamos todos os custos por nível para determinar o custo total de todos os níveis da recursão

Ideia geral

- ▶ uma árvore de recursão é melhor usada para obter um bom chute, que pode então ser provado pelo método da substituição
- ▶ quando usamos uma árvore de recursão para fazer um chute, podemos ignorar alguns detalhes
- ▶ se, por outro lado, formos cuidadosos no desenho da árvore de recursão e na soma de seus custos, podemos usá-la como uma prova direta da solução de uma recorrência
- ▶ em geral, optamos por usar a árvore de recursão para obter um bom chute da solução de uma recorrência e, em seguida, provamos que o chute está correto usando o método da substituição

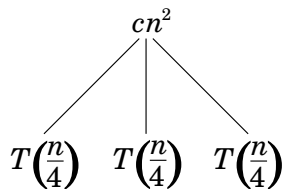
Exemplo

- ▶ vamos mostrar que a árvore de recursão fornece um bom chute para a recorrência $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$
- ▶ vamos focalizar na busca por um limitante superior para a solução
- ▶ em geral, pisos e tetos não importam muito na solução de recorrências e, por isso, vamos desenhar uma árvore de recursão para a recorrência $T(n) = 3T(n/4) + cn^2$, para alguma constante $c > 0$
- ▶ vamos supor por conveniência que n é uma potência de 4

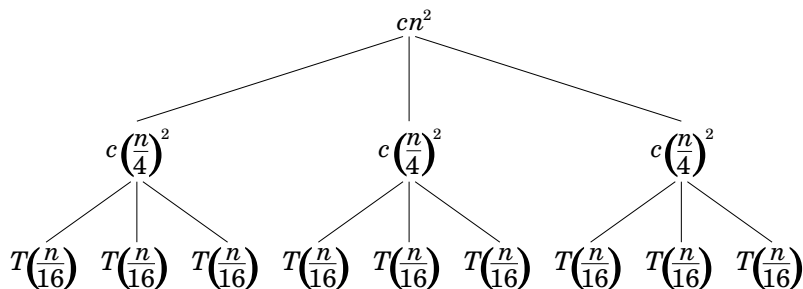
Exemplo

$$T(n)$$

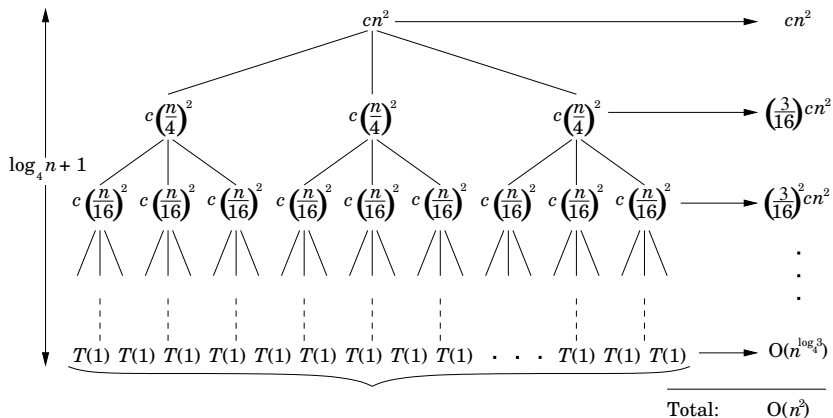
Exemplo



Exemplo



Exemplo



Exemplo

- ▶ observe que o tamanho de um subproblema decresce por um fator de 4 a cada vez que descemos um nível na árvore
- ▶ logo, certamente chegaremos a um caso pequeno
- ▶ para um vértice de profundidade i , o tamanho do subproblema associado a esse vértice é $n/4^i$
- ▶ dessa forma, o subproblema atinge tamanho 1 quando $n/4^i = 1$ ou equivalentemente quando $i = \log_4 n$
- ▶ portanto, a árvore tem $\log_4 n + 1$ níveis, com profundidades $0, 1, 2, \dots, \log_4 n$

Exemplo

- ▶ agora observe que um nível tem 3 vezes mais vértices que o nível acima e, assim, o número de vértices à profundidade i é 3^i
- ▶ como os tamanhos dos subproblemas diminuem por um fator de 4 para cada nível quando percorremos a árvore de cima para baixo a partir da raiz, então cada vértice à profundidade i tem um custo de $c(n/4^i)^2$, para $i = 0, 1, 2, \dots, \log_4 n - 1$
- ▶ assim, o custo total dos vértices à profundidade i é $3^i c(n/4^i)^2 = (3/16)^i cn^2$, para $i = 0, 1, 2, \dots, \log_4 n - 1$
- ▶ o último nível, onde se localizam as folhas à profundidade $\log_4 n$, tem $3^{\log_4 n} = n^{\log_4 3}$ vértices, cada um contribuindo com custo $T(1)$, o que implica em custo total $n^{\log_4 3} T(1)$, que é $\Theta(n^{\log_4 3})$

Exemplo

$$\begin{aligned}
 T(n) &= cn^2 + \left(\frac{3}{16}\right)cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\
 &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
 &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
 &= \left(\frac{1}{1 - (3/16)}\right) cn^2 + \Theta(n^{\log_4 3}) \\
 &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
 &= O(n^2).
 \end{aligned}$$

Exemplo

- ▶ então temos um chute $T(n) = O(n^2)$ para a recorrência original $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$
- ▶ agora podemos usar o método da substituição para verificar se nosso chute está correto
- ▶ queremos mostrar que $T(n) \leq dn^2$ para alguma constante $d > 0$

$$\begin{aligned}
 T(n) &\leq 3 T(\lfloor n/4 \rfloor) + cn^2 \\
 &\leq 3 d \lfloor n/4 \rfloor^2 + cn^2 \\
 &\leq 3 d (n/4)^2 + cn^2 \\
 &= \left(\frac{3}{16} \right) dn^2 + cn^2 \\
 &\leq dn^2,
 \end{aligned}$$

para $d \geq (16/13) c$.

Ideia geral

- ▶ o método mestre é uma receita para solucionar recorrências da forma

$$T(n) = aT(n/b) + f(n),$$

onde $a \geq 1$ e $b > 1$ são constantes e $f(n)$ é uma função assintoticamente positiva

- ▶ memorização de 3 casos

Método mestre

Teorema mestre

Sejam $a \geq 1$ e $b > 1$ constantes, $f(n)$ uma função e $T(n)$ definida sobre os inteiros não negativos pela recorrência

$$T(n) = aT(n/b) + f(n),$$

onde interpretamos n/b como $\lfloor n/b \rfloor$ ou $\lceil n/b \rceil$. Então $T(n)$ tem os seguintes limites assintóticos:

- (1) se $f(n) = O(n^{\log_b a - \varepsilon})$ para alguma constante $\varepsilon > 0$, então $T(n) = \Theta(n^{\log_b a})$.
- (2) se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \lg n)$.
- (3) se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ para alguma constante $\varepsilon > 0$ e se $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e todo n suficientemente grande, então $T(n) = \Theta(f(n))$.

Exemplos

Considere a recorrência $T(n) = 9T(n/3) + n$. Neste caso, temos $a = 9, b = 3, f(n) = n$ e assim temos que $n^{\log_b a} = n^{\log_3 9} = n^2$. Como $f(n) = n = O(n^{\log_3 9 - \varepsilon})$ para $\varepsilon = 1$, podemos aplicar o **caso 1** do teorema mestre e concluir que a solução é $T(n) = \Theta(n^2)$.

Exemplos

Considere a recorrência $T(n) = T(2n/3) + 1$. Neste caso, temos $a = 1, b = 3/2, f(n) = 1$ e assim temos que $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Como $f(n) = 1 = \Theta(n^{\log_b a}) = \Theta(1)$, o **caso 2** do teorema mestre se aplica e podemos concluir que é $T(n) = \Theta(\lg n)$.

Exemplos

Considere a recorrência $T(n) = 3T(n/4) + n \lg n$. Neste caso, temos $a = 3, b = 4, f(n) = n \lg n$ e $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. Como $f(n) = n \lg n = \Omega(n^{\log_4 3 + \varepsilon})$ para $\varepsilon \approx 0.2$, podemos aplicar o caso 3 do teorema mestre se a condição de regularidade for satisfeita. Para n suficientemente grande, temos que

$$af(n/b) = 3(n/4)\lg(n/4) \leq (3/4)n \lg n = cf(n)$$

para $c = 3/4$. Consequentemente, pelo **caso 3**, a recorrência é $T(n) = \Theta(n \lg n)$.

Exemplos

Considere a recorrência $T(n) = 2T(n/2) + n \lg n$. Neste caso, temos $a = 2, b = 2, f(n) = n \lg n$ e $n^{\log_b a} = n^{\log_2 2} = n$. Podemos pensar que o caso 3 se aplica, já que $f(n) = n \lg n$ é assintoticamente maior que $n^{\log_b a} = n$. O problema é que $f(n)$ não é *polinomialmente* maior que n . Ou seja, não conseguimos mostrar que $f(n) = n \lg n = \Omega(n^{1+\varepsilon})$ para qualquer $\varepsilon > 0$. Dessa forma, o teorema mestre não pode ser aplicado a esta recorrência.

Exercícios

- 4.1-1 Se o vetor $A[p..r]$ não tem elementos negativos, qual o valor de sua solidez? E se todos os elementos são negativos?
- 4.1-2 Escreva um pseudocódigo para o método ingênuo ou da força-bruta para solucionar o problema do segmento de soma máxima, com tempo de execução $\Theta(n^2)$. Mostre que seu algoritmo está correto.

Exercícios

- 4.1-5 Desenvolva um pseudocódigo não recursivo para o problema do segmento de soma máxima da seguinte forma. Faça uma varredura da esquerda para a direita no vetor de entrada, guardando os índices do subvetor cuja soma é máxima. Conhecendo um subvetor de $A[1..j]$ cuja soma é máxima, estenda a resposta encontrando um subvetor cuja soma é máxima terminando em $j+1$, usando a seguinte observação: um subvetor de soma máxima de $A[1..j+1]$ ou é um subvetor de soma máxima de $A[1..j]$ ou um subvetor $A[i..j+1]$, para algum $1 \leq i \leq j+1$. Determine um subvetor de soma máxima da forma $A[i..j+1]$ em tempo constante, baseado no conhecimento de um subvetor de soma máxima terminando no índice j . Analise o tempo de execução deste algoritmo.

Exercícios

- 4.3-1 Mostre que a solução de $T(n) = T(n-1) + n$ é $O(n^2)$.
- 4.3-2 Mostre que a solução de $T(n) = T(\lceil n/2 \rceil) + 1$ é $O(\lg n)$.
- 4.3-3 Vimos que a solução de $T(n) = 2T(\lfloor n/2 \rfloor) + n$ é $O(n \lg n)$. Mostre que a solução desta recorrência é também $\Omega(n \lg n)$. Conclua então que a solução é $\Theta(n \lg n)$.

Exercícios

4.3-5 Mostre que a solução da recorrência precisa do MERGESORT

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n), & \text{se } n > 1, \end{cases}$$

é $\Theta(n \lg n)$.

4.3-6 Mostre que a solução de $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$ é $O(n \lg n)$.

Exercícios

- 4.3-8 Usando o método mestre podemos mostrar que a solução da recorrência $T(n) = 4T(n/2) + n$ é $T(n) = \Theta(n^2)$. Mostre que uma tentativa de provar isso, usando o método da substituição com suposição que $T(n) \leq cn^2$, falha. Mostre então como subtrair um termo de pequena ordem que faça que a prova pelo método da substituição funcione.
- 4.3-9 Resolva a recorrência $T(n) = 3T(\sqrt{n}) + \lg n$ fazendo mudança de variáveis. Sua solução deve ser assintoticamente justa. Não se preocupe com problemas de arredondamento.

Exercícios

- 4.4-1 Use uma árvore de recursão para determinar um bom limitante superior assintótico para a recorrência $T(n) = 3T(\lfloor n/2 \rfloor) + n$. Use o método da substituição para verificar sua resposta.
- 4.4-2 Use uma árvore de recursão para determinar um bom limitante superior assintótico para a recorrência $T(n) = T(n/2) + n^2$. Use o método da substituição para verificar sua resposta.
- 4.4-3 Use uma árvore de recursão para determinar um bom limitante superior assintótico para a recorrência $T(n) = 4T(n/2 + 2) + n$. Use o método da substituição para verificar sua resposta.
- 4.4-4 Use uma árvore de recursão para determinar um bom limitante superior assintótico para a recorrência $T(n) = 2T(n-1) + 1$. Use o método da substituição para verificar sua resposta.
- 4.4-5 Use uma árvore de recursão para determinar um bom limitante superior assintótico para a recorrência $T(n) = T(n-1) + T(n/2) + n$. Use o método da substituição para verificar sua resposta.

Exercícios

4.4-6 Argumente que a solução da recorrência

$T(n) = T(n/3) + T(2n/3) + cn$, onde c é uma constante, é $\Omega(n \lg n)$ usando uma árvore de recursão.

4.4-7 Desenhe uma árvore de recursão para $T(n) = 4T(\lfloor n/2 \rfloor) + cn$, onde c é uma constante, e forneça um limitante assintótico justo para sua solução. Verifique seu limitante com o método da substituição.

4.4-8 Use uma árvore de recursão para determinar uma solução assintótica justa para a recorrência $T(n) = T(n-a) + T(a) + cn$, onde $a \geq 1$ e $c > 0$ são constantes.

4.4-9 Use uma árvore de recursão para determinar uma solução assintótica justa para a recorrência $T(n) = T(\alpha n) + T((1-\alpha)n) + cn$, onde α é uma constante no intervalo $0 < \alpha < 1$ e $c > 0$ é também uma constante.

Exercícios

4.5-1 Use o teorema mestre para fornecer limitantes assintóticos justos para as seguintes recorrências:

1. $T(n) = 2T(n/4) + 1$.
2. $T(n) = 2T(n/4) + \sqrt{n}$.
3. $T(n) = 2T(n/4) + n$.
4. $T(n) = 2T(n/4) + n^2$.

4.5-3 Use o teorema mestre para mostrar que a solução para a recorrência da busca binária $T(n) = T(n/2) + \Theta(1)$ é $T(n) = \Theta(\lg n)$.

4.5-4 Podemos aplicar o teorema mestre na recorrência $T(n) = 4T(n/2) + n^2 \lg n$? Justifique sua resposta. Forneça um limitante superior assintótico para esta recorrência.

Problemas

4-1 Exemplos de recorrências

Forneça limitantes inferior e superior para $T(n)$ em cada uma das seguintes recorrências. Considere que $T(n)$ é constante para $n \leq 2$. Faça seus limitantes tão justos quanto possível e justifique suas respostas.

- (a) $T(n) = 2T(n/2) + n^4$.
- (b) $T(n) = T(7n/10) + n$.
- (c) $T(n) = 16T(n/4) + n^2$.
- (d) $T(n) = 7T(n/3) + n^2$.
- (e) $T(n) = 7T(n/2) + n^2$.
- (f) $T(n) = 2T(n/4) + \sqrt{n}$.
- (g) $T(n) = T(n-2) + n^2$.

Problemas

4-3 Mais exemplos de recorrências

Forneça limitantes inferior e superior para $T(n)$ em cada uma das seguintes recorrências. Considere que $T(n)$ é constante para n suficientemente pequeno. Faça seus limitantes tão justos quanto possível e justifique suas respostas.

- (a) $T(n) = 4T(n/3) + n \lg n.$
- (b) $T(n) = 3T(n/3) + n/\lg n.$
- (c) $T(n) = 4T(n/2) + n^2 \sqrt{n}.$
- (d) $T(n) = 3T(n/3 - 2) + n/2.$
- (e) $T(n) = 2T(n/2) + n/\lg n.$
- (f) $T(n) = T(n/2) + T(n/4) + T(n/8) + n.$
- (g) $T(n) = T(n-1) + 1/n.$
- (h) $T(n) = T(n-1) + \lg n.$
- (i) $T(n) = T(n-2) + 1/\lg n.$
- (j) $T(n) = \sqrt{n} T(\sqrt{n}) + n.$